Volume 1, Issue 1, pp. xx-xx, 2026. *ISSN (Online): 3106-1435*

Recursive Multi Agent Tiny Recursive Models for Efficient Latent Space Reasoning

Panashe Chiurunge
Deep Analytics, Harare, Zimbabwe
Corresponding author email to be inserted

Abstract: Tiny recursive models show that a compact network can solve difficult structured reasoning tasks by repeatedly refining a latent state and a candidate answer. Their homogeneous recursion, however, applies the same transformation at every reasoning step, provides limited mechanisms for functional specialization, and may spend equal computation on easy and difficult instances. This paper proposes a recursive multi agent tiny recursive model in which several tiny specialized reasoners collaborate through continuous latent representations. The framework replaces a single recursive network with role conditioned agents, introduces private agent states and a shared workspace, uses residual latent links for communication, and allocates additional recursion when agent predictions disagree. Sparse routing activates only the agents required by the current state, while cell level recursive attention concentrates updates on uncertain output locations. Four collaboration topologies are formalized, namely sequential, mixture, deliberation, and distillation. An inner outer optimization procedure assigns local role competence and global system credit. A composite objective balances answer accuracy, iterative improvement, verification, specialization, consensus, routing balance, computation, and halting. The recommended MA TRM Lite architecture preserves a shared two layer backbone and adds low rank role adapters, lightweight latent links, a verifier, and a dynamic router. We derive parameter and computation bounds, present sufficient conditions for stable recursion, provide executable algorithms, and define a rigorous evaluation protocol for Sudoku, maze, and abstraction reasoning benchmarks. The resulting framework is a testable research proposal and does not claim empirical superiority before controlled parameter matched and compute matched evaluation.

Keywords: adaptive computation: agent specialization: latent communication: multi agent reasoning: recursive neural networks: sparse routing: tiny models

1 Introduction

Large language models have made substantial progress on reasoning, but their computational cost and dependence on explicit token generation motivate alternative architectures for compact, task specific reasoning. The Abstraction and Reasoning Corpus, Sudoku, and path finding tasks expose a useful regime in which the required output is structured, exact, and often small relative to the internal computation needed to infer it [1]. Hierarchical Reasoning Models introduced recurrent computation at multiple temporal scales [2]. Tiny Recursive Models, abbreviated TRM, simplified this direction to a single small network that repeatedly updates a latent state and a candidate answer [3]. The published TRM results demonstrate that repeated computation can be a scaling axis distinct from parameter count.

TRM also raises a structural question. If one tiny transformation is useful when applied repeatedly, can a small collection of complementary transformations provide a stronger recursion operator without losing parameter efficiency? Recursive Multi-Agent Systems answer a related question for heterogeneous language model agents by connecting their hidden states through trainable RecursiveLink modules and optimizing the resulting collaboration loop as one differentiable system [4]. The present work transfers the underlying principle from billion parameter language agents to tiny non-autoregressive reasoners.

We propose the Recursive Multi-Agent Tiny Recursive Model, abbreviated MA-TRM. Several tiny specialized agents collaboratively refine a shared solution in latent space. The proposed model is designed around eleven principles:

1) replace the single recursive network with specialized tiny agents;
2) communicate through latent states rather than decoded intermediate answers;
3) maintain private agent states alongside a shared workspace;
4) use disagreement to allocate recursion;
5) route computation sparsely and dynamically;
6) support sequential, mixture, deliberation, and distillation topologies;
7) train through inner and outer co-optimization;
8) optimize a multi-term system objective;
9) focus recursion on uncertain cells;
10) preserve parameter efficiency through controlled sharing;
11) instantiate these ideas in MA-TRM-Lite.

The paper is architectural and methodological. It defines a falsifiable model family and an evaluation protocol. It deliberately avoids reporting synthetic results or implying empirical gains that have not been measured. This distinction is important because analyses of TRM on ARC suggest that augmentation, task identity conditioning, and test-time voting can materially influence reported accuracy [5]. Any credible evaluation of MA-TRM must therefore isolate architectural gains from parameter count, compute, augmentation, and identity features.

The principal contributions are as follows. First, we formulate a multi-agent recursive state space with role specialization, private memory, and a shared latent workspace. Second, we define low rank latent links, sparse routing, disagreement-driven halting, and cell-level attention in a common differentiable framework. Third, we formalize four collaboration topologies and a two-

stage co-optimization algorithm. Fourth, we provide parameter, runtime, and stability analyses. Fifth, we specify MA-TRM-Lite as the recommended first implementation and define a complete benchmark plan.

2 Background and Motivation

2.1 Tiny Recursive Models

Let an input instance be x, the candidate discrete answer at supervision step t be $\hat{y}_t$, its embedding be y_t, and the latent reasoning state be z_t. A simplified TRM update applies the same tiny network repeatedly:

$$z_{t,j+1} = f_\theta(x, y_t, z_{t,j}), \quad j = 0, \ldots, n-1, \tag{1}$$

$$y_{t+1} = f_\theta(y_t, z_{t,n}), \tag{2}$$

where n is the number of latent updates inside a deep recursion step. Several deep recursion steps may be executed before the answer head and supervision loss are applied. The released TRM configuration uses a two-layer tiny network and reports strong results with a core network of approximately seven million parameters [3]. The architecture is non-autoregressive, which makes it computationally attractive for structured outputs.

The central advantage of (1) and (2) is parameter reuse. Effective depth grows with the number of recursive evaluations, while the number of learned transformation parameters remains fixed. Its central restriction is homogeneous recursion. The same operator must generate hypotheses, perform transformations, detect errors, and determine completion.

A further concern is that recursive depth does not automatically imply effective iterative reasoning. An independent analysis found that a substantial part of ARC performance can depend on test-time voting and task identity conditioning, and that accuracy may saturate after a small number of recursion steps [5]. Probabilistic TRM addresses deterministic convergence by injecting noise into recursive trajectories and selecting among candidates with the existing quality head [6]. These findings motivate explicit diversity, disagreement, and adaptive computation within the recursive model itself.

2.2 Recursive Multi-Agent Systems

RecursiveMAS models a multi-agent system as a latent recursion loop. Each agent generates latent thoughts, an inner link maps the agent's last-layer states back into its input embedding space, and an outer link transfers states across heterogeneous agents [4]. A residual link has the form

$$R(h) = Ph + W_2\sigma(W_1 h), \tag{3}$$

where P is either the identity or a dimension-matching projection. The approach supports four collaboration patterns and uses an inner stage to align latent generation, followed by an outer stage that optimizes the complete recursive system.

The proposed MA-TRM differs in four important respects. First, the agents are tiny non-autoregressive reasoners, rather than independent language models. Second, the answer is a structured tensor, such as a grid or sequence, rather than text.

Table 1. Principal notation.

Symbol	Meaning
M	number of specialized agents
$T_{\max}$	maximum collaboration rounds
k_t	active agents in round t
$Z_t^{(i)}$	private state of agent i
S_t	shared latent workspace
R_{ij}	latent link from agent i to j
$P_t^{(i)}$	agent i output distribution
D_t	system disagreement score
$U_{t,q}$	uncertainty at cell q
A_t	set of active agents
q_t	halting probability

Third, all agents can share a backbone and differ through low rank role adapters. Fourth, disagreement is used as a direct control signal for routing, spatial attention, and recursion depth.

2.3 Research Gap

TRM demonstrates homogeneous recursive reuse, and RecursiveMAS demonstrates latent collaboration among heterogeneous language agents. Neither directly studies the regime in which multiple tiny specialists share most parameters, preserve private states, communicate continuously, route sparsely, and refine a structured answer at cell resolution. MA-TRM targets this gap.

3 Problem Formulation

Consider a supervised structured reasoning dataset

$$\mathcal{D} = \{(x^{(n)}, y^{\star(n)})\}_{n=1}^{N}, \tag{4}$$

where x is an input structure and $y^\star \in \{1, \ldots, C\}^L$ is a target sequence of L categorical cells. A grid is flattened into $L = H \times W$ cells. The model maintains:

- input embeddings $X = E_x(x) \in \mathbb{R}^{L\times d}$;
- an embedded candidate answer $Y_t = E_y(\hat{y}_t) \in \mathbb{R}^{L\times d}$;
- M private agent states $Z_t^{(i)} \in \mathbb{R}^{L\times d}$;
- a shared workspace $S_t \in \mathbb{R}^{L\times d}$;
- per-agent predictive distributions $P_t^{(i)} \in [0,1]^{L\times C}$.

The objective is to learn a recursive system that produces $\hat{y}_T = y^\star$ with high exact accuracy while minimizing active agents, recursion rounds, parameters, and energy.

4 Recursive Multi-Agent Architecture

4.1 Specialized Tiny Agents

Let the agent set be $\mathcal{A} = \{A_1, \ldots, A_M\}$. In the recommended four-agent design, the roles are pattern discovery, transformation, criticism, and verification. Each agent receives a role embedding e_i and a role-specific adapter while sharing a common tiny backbone F_Θ.

For a backbone layer with hidden input H, agent i applies

$$\Delta_i(H) = B_i\,\sigma(A_i\,\mathrm{LN}(H))\,, \tag{5}$$

$$F_i(H) = F_\Theta(H) + \alpha_i\Delta_i(H), \tag{6}$$

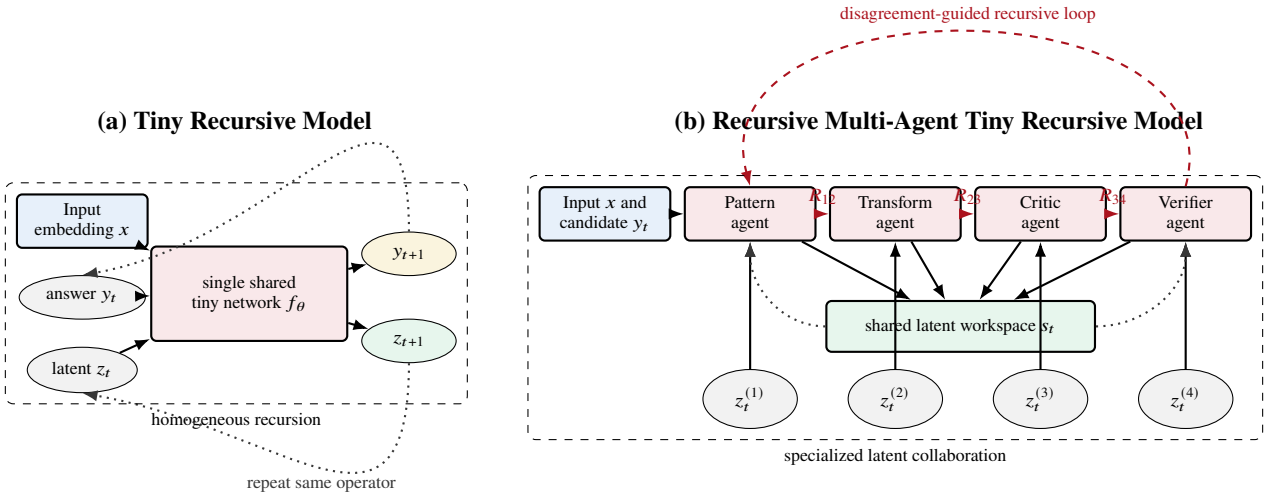

Figure 1. Architectural distinction. TRM repeatedly applies one operator to the answer and latent state. MA-TRM composes role-specialized updates, preserves private states, communicates through latent links, and maintains a shared workspace.

where $A_i \in \mathbb{R}^{r_a \times d}$, $B_i \in \mathbb{R}^{d \times r_a}$, $r_a \ll d$, and α_i is a learned scale. The complete input to agent i at round t is

$$H_t^{(i)} = \Psi_i\left(X, Y_t, Z_t^{(i)}, S_t, M_t^{(i)}, e_i\right), \tag{7}$$

where $M_t^{(i)}$ is the incoming latent message and Ψ_i is a lightweight fusion operator, implemented by summation after normalization or by a gated projection.

The private state update is

$$G_t^{(i)} = \text{sigmoid}\left(W_g^{(i)}\,\text{LN}(H_t^{(i)}) + b_g^{(i)}\right), \tag{8}$$

$$\widetilde{Z}_{t+1}^{(i)} = F_i(H_t^{(i)}), \tag{9}$$

$$Z_{t+1}^{(i)} = G_t^{(i)} \odot \widetilde{Z}_{t+1}^{(i)} + \left(1 - G_t^{(i)}\right) \odot Z_t^{(i)}. \tag{10}$$

The gate reduces destructive overwriting and allows an agent to retain useful role-specific evidence across rounds.

4.2 Latent Communication

Decoded communication would require each agent to project hidden states to output classes, make a discrete or soft decision, and re-embed that decision for the next agent. MA-TRM instead uses residual low rank latent links:

$$R_{ij}(H) = P_{ij}H + B_{ij}\sigma\left(A_{ij}\,\text{LN}(H)\right), \tag{11}$$

where $A_{ij} \in \mathbb{R}^{r_\ell \times d_i}$, $B_{ij} \in \mathbb{R}^{d_j \times r_\ell}$, and P_{ij} is the identity when dimensions match. The message to agent j is

$$M_t^{(j)} = R_{ij}\left(Z_{t+1}^{(i)}\right). \tag{12}$$

The residual branch preserves source semantics, and the low rank branch learns role alignment. Unlike text communication, the link remains differentiable and does not pass through an argmax bottleneck.

4.3 Agent-Specific States and Shared Workspace

Private states preserve incompatible or complementary hypotheses. A shared workspace integrates evidence needed by all agents. Let the active set at round t be A_t. The workspace attention scores are

$$e_t^{(i)} = w_s^\top \tanh\left(W_z\,\text{Pool}(Z_{t+1}^{(i)}) + W_s\,\text{Pool}(S_t)\right), \tag{13}$$

$$\beta_t^{(i)} = \frac{\exp(e_t^{(i)})}{\sum_{j \in A_t}\exp(e_t^{(j)})}. \tag{14}$$

The integrated proposal is

$$\widetilde{S}_{t+1} = \sum_{i \in A_t} \beta_t^{(i)} P_i Z_{t+1}^{(i)}, \tag{15}$$

and the gated workspace update is

$$\Gamma_t = \text{sigmoid}\left(W_\gamma[\text{LN}(S_t); \text{LN}(\widetilde{S}_{t+1})]\right), \tag{16}$$

$$S_{t+1} = \Gamma_t \odot S_t + (1 - \Gamma_t) \odot \widetilde{S}_{t+1}. \tag{17}$$

The answer head receives both the previous candidate and the updated workspace:

$$L_{t+1} = W_o\,\text{LN}(Y_t + S_{t+1}) + b_o, \tag{18}$$

$$\bar{P}_{t+1} = \text{softmax}(L_{t+1}), \tag{19}$$

$$\hat{y}_{t+1,q} = \arg\max_c \bar{P}_{t+1,q,c}. \tag{20}$$

4.4 Disagreement-Driven Recursion

Each active agent produces a provisional output distribution

$$P_t^{(i)} = \text{softmax}\left(W_o^{(i)}\,\text{LN}(Z_{t+1}^{(i)} + S_t)\right). \tag{21}$$

The weighted mean prediction is

$$\bar{P}_t = \sum_{i \in A_t} \omega_t^{(i)} P_t^{(i)}, \quad \sum_{i \in A_t} \omega_t^{(i)} = 1. \tag{22}$$

We define system disagreement by the weighted Jensen-Shannon divergence:

$$D_t = \frac{1}{L} \sum_{q=1}^{L} \sum_{i \in A_t} \omega_t^{(i)} \, \mathrm{KL}\Big(P_{t,q}^{(i)} \,\|\, \bar{P}_{t,q} \Big). \tag{23}$$

The verifier estimates correctness confidence $C_t \in [0, 1]$, while answer change is

$$\Delta_t = \frac{1}{L} \sum_{q=1}^{L} \mathbb{I}[\hat{y}_{t,q} \neq \hat{y}_{t-1,q}]. \tag{24}$$

The learned halting probability is

$$q_t = \mathrm{sigmoid}\big(w_h^\top [\mathrm{Pool}(S_t); D_t; C_t; \Delta_t; t/T_{\max}] + b_h \big). \tag{25}$$

Inference halts when $q_t \geq \tau_h$, $D_t \leq \tau_D$, $C_t \geq \tau_C$, and $\Delta_t \leq \tau_\Delta$, or when $t = T_{\max}$. This conjunctive rule prevents high confidence alone from terminating an unstable collaboration.

Adaptive computation is related to Adaptive Computation Time [7], but the control signal here includes multi-agent disagreement and structured answer stability. The model does not assume convergence to a fixed point. It uses a bounded number of rounds and learns a stopping policy.

4.5 Sparse Dynamic Routing

A router activates only a subset of agents:

$$r_t = \mathrm{softmax}(W_r \, \mathrm{Pool}([X; Y_t; S_t]) + b_r), \tag{26}$$

$$A_t = \mathrm{TopK}(r_t, k_t). \tag{27}$$

During training, a straight-through top-k estimator or a Gumbel relaxation can preserve approximate gradients [10]. The active agent output is weighted by renormalized gate probabilities. Sparse routing follows the general principle of conditional computation and mixture-of-experts models [8, 9], but routing operates over semantic reasoning roles rather than large feed-forward experts.

To prevent expert collapse, the utilization fraction u_i and mean router probability v_i are regularized:

$$\mathcal{L}_{\mathrm{bal}} = M \sum_{i=1}^{M} u_i v_i. \tag{28}$$

A low value is obtained when workload is spread across roles. A separate entropy schedule permits broad exploration early and sharper routing later.

4.6 Cell-Level Recursive Attention

Global recursion can repeatedly modify cells that are already correct. For each cell q, define predictive uncertainty and disagreement:

$$U_{t,q} = 1 - \max_c \bar{P}_{t,q,c}, \tag{29}$$

$$D_{t,q} = \sum_{i \in A_t} \omega_t^{(i)} \, \mathrm{KL}\Big(P_{t,q}^{(i)} \,\|\, \bar{P}_{t,q} \Big). \tag{30}$$

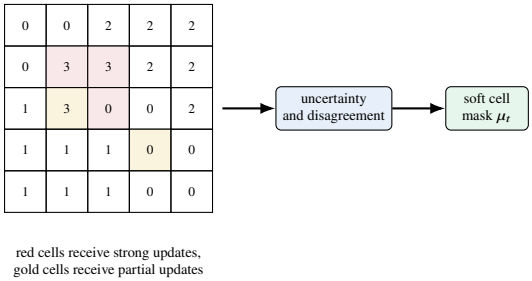

Figure 2. Cell-level recursive attention. Computation concentrates on unstable output locations while preserving a periodic global refresh.

A differentiable attention mask is

$$\mu_{t,q} = \mathrm{sigmoid}\Big(\tau^{-1} \big[a_D D_{t,q} + a_U U_{t,q} + a_\Delta \mathbb{I}[\hat{y}_{t,q} \neq \hat{y}_{t-1,q}] - \tau_m \big] \Big). \tag{31}$$

The next state update is spatially gated:

$$Z_{t+1,q}^{(i)} = \mu_{t,q} \widetilde{Z}_{t+1,q}^{(i)} + (1 - \mu_{t,q}) Z_{t,q}^{(i)}. \tag{32}$$

A periodic global refresh, for example every second round, sets $\mu_{t,q} = 1$ for all cells. This prevents early confidence errors from becoming permanently frozen.

5 Collaboration Topologies

The same agent modules can be connected through four topologies. Let Φ_t denote one complete collaboration round.

5.1 Sequential Topology

Agents form an ordered chain:

$$\Phi_t^{\mathrm{seq}} = A_M \circ R_{M-1,M} \circ A_{M-1} \circ \cdots \circ R_{1,2} \circ A_1. \tag{33}$$

For grid reasoning, a natural sequence is pattern agent, transformation agent, critic, verifier. This topology has low coordination overhead and forms the basis of MA-TRM-Lite.

5.2 Mixture Topology

Agents operate in parallel and an integrator combines their states:

$$H_t^{(i)} = A_i(X, Y_t, Z_t^{(i)}, S_t), \tag{34}$$

$$\Phi_t^{\mathrm{mix}} = A_{\mathrm{int}} \left(\sum_{i \in A_t} \beta_t^{(i)} R_{i,\mathrm{int}}(H_t^{(i)}) \right). \tag{35}$$

This topology is suitable when multiple transformation hypotheses should be explored independently.

5.3 Deliberation Topology

A proposer and a critic alternate until the verifier accepts a revised state:

$$H_t^p = A_p(S_t), \tag{36}$$

$$H_t^c = A_c(R_{pc}(H_t^p), S_t), \tag{37}$$

$$H_t^r = A_r(R_{cr}(H_t^c), H_t^p), \tag{38}$$

Table 2. Properties of the four collaboration topologies.

Topology	Benefit and principal risk	Suggested use
Sequential	low overhead and clear roles, but order sensitive	first implementation
Mixture	parallel hypotheses, but higher integration cost	ambiguous transformations
Deliberation	explicit correction, but possible oscillation	difficult uncertain tasks
Distillation	low cost deployment, but teacher bias	compressed inference

where A_r may share the proposer backbone with a revision role adapter. Deliberation allocates more compute to ambiguous tasks but can create oscillation, so the answer-change term in (25) is essential.

5.4 Distillation Topology

A stronger collaborative teacher supervises a compact student:

$$\mathcal{L}_{\text{distill}} = T_d^2 \,\mathrm{KL}\Big(P_T^{(T_d)} \,\|\, P_S^{(T_d)}\Big) + \lambda_z \|P_z S_T - Z_S\|_2^2, \tag{39}$$

where T_d is the distillation temperature. The student can be a standard TRM or a two-agent MA-TRM. Distillation separates expensive training-time collaboration from low-cost deployment.

6 Inner-Outer Co-Optimization

6.1 Inner Role Learning

The inner stage gives each agent a measurable role competence before full system training. The role losses can be generated from structured corruptions and transformations:

$$\mathcal{L}_{\text{pattern}} = \mathrm{CE}(\hat{r}, r^\star), \tag{40}$$
$$\mathcal{L}_{\text{transform}} = \mathrm{CE}(P^{\text{tr}}, y^\star), \tag{41}$$
$$\mathcal{L}_{\text{critic}} = \mathrm{BCE}(c, \mathbb{I}[\tilde{y} = y^\star]), \tag{42}$$
$$\mathcal{L}_{\text{verify}} = \mathrm{BCE}(v, \mathbb{I}[\hat{y} = y^\star]). \tag{43}$$

Here $r^\star$ denotes an available synthetic relation label, and $\tilde{y}$ is a corrupted candidate. When explicit relation labels are unavailable, the pattern agent can use contrastive agreement between augmented views.

Let θ collect shared backbone and adapter parameters, and let ϕ collect latent links, router, workspace, attention, and halting parameters. A one-step bilevel form is

$$\theta'(\phi) = \theta - \eta \nabla_\theta \mathcal{L}_{\text{inner}}(\theta, \phi; \mathcal{B}_{\text{in}}), \tag{44}$$
$$\phi \leftarrow \phi - \beta \nabla_\phi \mathcal{L}_{\text{outer}}(\theta'(\phi), \phi; \mathcal{B}_{\text{out}}). \tag{45}$$

In practice, exact second-order differentiation is optional. A stable implementation alternates several inner updates with one outer update, then performs a final joint fine-tuning stage.

6.2 Outer System Learning

The outer stage unrolls the complete system for up to $T_{\max}$ rounds and assigns global credit to all active links and agents. Let $\ell_t = \mathrm{CE}(\bar{P}_t, y^\star)$. The proposed objective is

$$\begin{aligned}\mathcal{L} = {} & \lambda_f \ell_T + \lambda_s \sum_{t=1}^{T} w_t \ell_t + \lambda_{\text{imp}} \mathcal{L}_{\text{imp}} + \lambda_v \mathcal{L}_{\text{ver}} \\ & + \lambda_{\text{spec}} \mathcal{L}_{\text{spec}} + \lambda_{\text{cons}} \mathcal{L}_{\text{cons}} + \lambda_{\text{bal}} \mathcal{L}_{\text{bal}} \\ & + \lambda_c \mathcal{L}_{\text{compute}} + \lambda_h \mathcal{L}_{\text{halt}} + \lambda_m \mathcal{L}_{\text{mask}}.\end{aligned} \tag{46}$$

The terms are defined below.

Iterative improvement penalizes rounds that fail to reduce answer loss:

$$\mathcal{L}_{\text{imp}} = \sum_{t=2}^{T} \max(0, \ell_t - \ell_{t-1} + \delta). \tag{47}$$

Verifier calibration uses both correctness and a Brier term:

$$\mathcal{L}_{\text{ver}} = \mathrm{BCE}(C_t, c_t^\star) + \lambda_B (C_t - c_t^\star)^2, \tag{48}$$

where $c_t^\star = \mathbb{I}[\hat{y}_t = y^\star]$ for exact verification, or a normalized cell accuracy target.

Role specialization discourages identical private states:

$$\mathcal{L}_{\text{spec}} = \frac{1}{M(M-1)} \sum_{i \neq j} \Big|\cos(\widetilde{z}^{(i)}, \widetilde{z}^{(j)})\Big|, \tag{49}$$

where $\widetilde{z}^{(i)}$ is a centered pooled state. Output consensus is applied more strongly in later rounds:

$$\mathcal{L}_{\text{cons}} = \sum_{t=1}^{T} \rho_t \sum_{i \in A_t} \omega_t^{(i)} \,\mathrm{KL}(P_t^{(i)} \| \bar{P}_t), \quad \rho_{t+1} \geq \rho_t. \tag{50}$$

This schedule permits early hypothesis diversity and late agreement.

Expected computation is regularized by

$$\mathcal{L}_{\text{compute}} = \frac{1}{T_{\max} M} \sum_{t=1}^{T_{\max}} (1 - q_{t-1}) |A_t|, \tag{51}$$

with $q_0 = 0$. Halting supervision targets the first correct and stable round $t^\dagger$:

$$\mathcal{L}_{\text{halt}} = \sum_{t=1}^{T_{\max}} \mathrm{BCE}(q_t, \mathbb{I}[t \geq t^\dagger]). \tag{52}$$

The mask sparsity loss is

$$\mathcal{L}_{\text{mask}} = \frac{1}{TL} \sum_{t,q} \mu_{t,q}, \tag{53}$$

subject to a minimum update fraction and periodic global refresh.

7 MA-TRM-Lite Recommended Architecture

MA-TRM-Lite is intended as the first reproducible implementation. It uses one shared two-layer backbone, four role adapters, sequential latent collaboration, a shared workspace, top-k routing, a verifier, and at most four collaboration rounds.

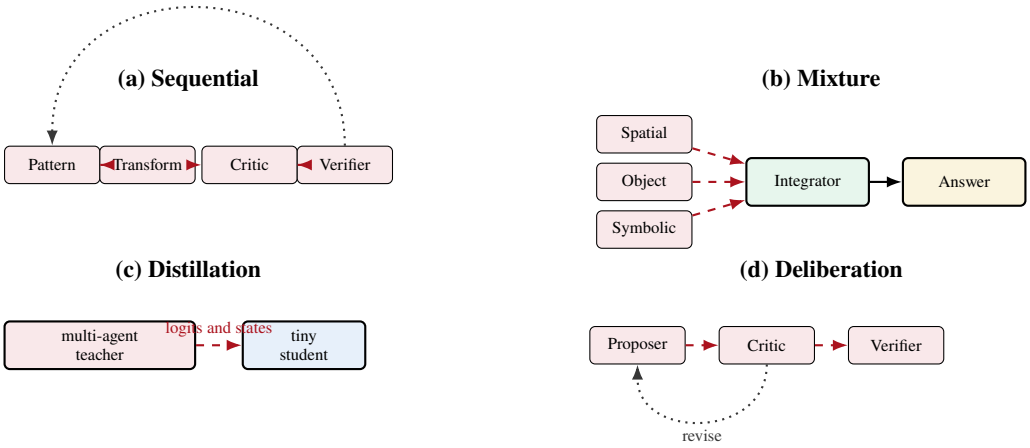

Figure 3. Four collaboration topologies supported by MA-TRM. The topologies use the same latent-link interface and can be compared under matched parameter and computation budgets.

Algorithm 1 MA-TRM inference with sparse routing and disagreement-driven recursion

Require: Input x, maximum rounds $T_{\max}$, agents $\{A_i\}_{i=1}^{M}$

$X \leftarrow E_x(x)$, initialize $\hat{y}_0$, Y_0, S_0, $\{Z_0^{(i)}\}$
for $t = 0$ to $T_{\max} - 1$ **do**
  $A_t \leftarrow \mathrm{TopK}(\mathrm{Router}(X, Y_t, S_t), k_t)$
  compute cell mask μ_t from uncertainty, disagreement, and change
  for each active agent i in topology order **do**
    $M_t^{(i)} \leftarrow R_{ji}(Z_{t+1}^{(j)})$ if predecessor j exists
    $\widetilde{Z}_{t+1}^{(i)} \leftarrow A_i(X, Y_t, Z_t^{(i)}, S_t, M_t^{(i)})$
    $Z_{t+1}^{(i)} \leftarrow \mu_t \odot \widetilde{Z}_{t+1}^{(i)} + (1 - \mu_t) \odot Z_t^{(i)}$
    $P_t^{(i)} \leftarrow \mathrm{Head}_i(Z_{t+1}^{(i)}, S_t)$
  end for
  $S_{t+1} \leftarrow \mathrm{WorkspaceUpdate}(S_t, \{Z_{t+1}^{(i)}\})$
  $(\bar{P}_{t+1}, \hat{y}_{t+1}) \leftarrow \mathrm{AnswerHead}(Y_t, S_{t+1})$
  compute D_{t+1}, C_{t+1}, Δ_{t+1}, and q_{t+1}
  if $q_{t+1} \geq \tau_h$ and $D_{t+1} \leq \tau_D$ and $C_{t+1} \geq \tau_C$ and $\Delta_{t+1} \leq \tau_\Delta$ **then**
    break
  end if
  $Y_{t+1} \leftarrow E_y(\hat{y}_{t+1})$
end for
return $\hat{y}_{t+1}$, diagnostics, and recursion trace

7.1 Agent Roles

A1. Pattern agent: extracts objects, symmetries, repetitions, color relations, and spatial correspondences.
A2. Transformation agent: maps detected relations into a candidate output transformation.
A3. Critic agent: compares the candidate with demonstrations, detects contradictions, and identifies suspect cells.
A4. Verifier agent: estimates exact correctness, cell correctness, and whether another round is beneficial.

The router may skip the critic or pattern agent on simple instances, but the verifier always runs before halting.

Algorithm 2 Inner-outer co-optimization

Require: Training data $\mathcal{D}$, inner steps K, outer rounds $T_{\max}$

initialize shared backbone, role adapters, links, router, workspace, and heads
for each training cycle **do**
  for $k = 1$ to K **do**
    sample $\mathcal{B}_{\text{in}}$ and construct role-specific corruptions
    update backbone and role adapters using (43)
    align latent links by state reconstruction or contrastive transfer
  end for
  sample $\mathcal{B}_{\text{out}}$
  unroll Algorithm 1 without hard stopping
  compute complete objective in (46)
  update links, router, workspace, attention, halting, and selected adapter parameters
  periodically unfreeze the shared backbone for joint low-rate refinement
end for
calibrate verifier and halting thresholds on a held-out validation set

7.2 Round Computation

For a sequential active path $\pi_t = (i_1, \ldots, i_{k_t})$, one round is

$$H_t^{(i_{j+1})} = A_{i_{j+1}}\left(R_{i_j i_{j+1}}(H_t^{(i_j)}), X, Y_t, Z_t^{(i_{j+1})}, S_t\right). \tag{54}$$

After the verifier, the shared workspace and answer are updated. The verifier's state is linked back to the first active agent for the next round.

7.3 Default Configuration

Table 3 gives a conservative starting point. These values are hypotheses for controlled experiments, rather than final tuned values.

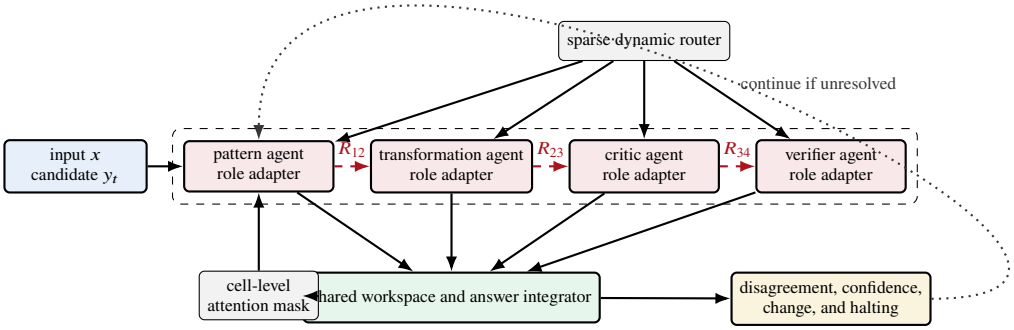

Figure 4. Recommended MA-TRM-Lite architecture. Dashed red arrows are continuous latent messages. Solid arrows update the shared workspace. Each role uses a low rank adapter attached to one shared backbone. The router selects active roles, the cell mask controls spatial computation, and the verifier controls recursion.

Table 3. Recommended initial MA-TRM-Lite configuration.

Component	Initial value
Shared backbone	2 layers, width 512
Agents	4 role-conditioned agents
Role adapter rank	$r_a = 8$
Latent-link rank	$r_\ell = 8$
Maximum rounds	$T_{\max} = 4$
Active roles	top 2, verifier mandatory
Workspace	gated weighted average
Cell mask	soft training, hard inference
Precision	bfloat16 where supported
Optimization	AdamW with EMA

8 Parameter, Runtime, and Stability Analysis

8.1 Controlled Parameter Sharing

Assume a shared backbone with P_Θ parameters, L_b adapted layers, hidden width d, role adapter rank r_a, M agents, E latent links, and link rank r_ℓ. Ignoring biases and small heads, the total parameter count is

$$P_{\text{MA-TRM}} = P_\Theta + 2ML_b dr_a + 2Edr_\ell + P_{\text{ctrl}}, \tag{55}$$

where P_{ctrl} includes routing, workspace, verifier, and answer heads.

Proposition 1 (Parameter overhead)**.** *If $r_a, r_\ell = o(d)$ and M, E, L_b are fixed, then the additional parameters of MA-TRM are $o(P_\Theta)$ for a backbone whose dense layers scale as $\Omega(L_b d^2)$.*

Proof. The adapter and link overhead in (55) is $O(L_b Mdr_a + Edr_\ell)$. The dense backbone is $\Omega(L_b d^2)$. Their ratio is $O(Mr_a/d + Er_\ell/(L_b d))$, which approaches zero when the ranks grow more slowly than d. □

For $d = 512$, $L_b = 2$, $M = 4$, $E = 4$, and $r_a = r_\ell = 8$, the role adapters and latent links add approximately 98,304 matrix parameters before small control heads. This illustrative calculation indicates that role specialization need not multiply a seven million parameter backbone by four.

Parameter reporting must distinguish the core network, role modules, puzzle identity embeddings, and all other trainable states. This requirement responds directly to concerns that task identity embeddings can materially affect the effective parameter count and generalization interpretation of ARC systems [5].

8.2 Sparse Compute Bound

Let one backbone evaluation cost C_A, one latent link cost C_R, workspace and control cost C_C, and let $k_t = |A_t|$.

Proposition 2 (Sparse round cost)**.** *The cost of an adaptive MA-TRM execution is*

$$C_{\text{MA-TRM}} = \sum_{t=1}^{T} (k_t C_A + (k_t - 1)C_R + C_C). \tag{56}$$

Relative to a dense M-agent execution with the same number of rounds, the agent-compute ratio is $\sum_t k_t/(TM)$.

Proof. Each active agent incurs one backbone evaluation, each adjacent active pair incurs one link, and each round incurs one control update. Dividing the active agent evaluations by the dense count TM gives the ratio. □

For top-2 routing with four agents, the dominant agent computation is approximately one half of dense four-agent collaboration, before router and verifier overhead. This is a computational accounting statement, not a speed claim. Hardware utilization and kernel launch overhead must be measured.

8.3 Latent Communication Cost

Let C be the number of output classes, d the latent width, and r_ℓ the link rank. A decoded handoff requires at least an output projection of order $O(LCd)$ and an embedding operation. A low rank latent link costs $O(Ldr_\ell)$, plus an identity residual.

Proposition 3 (Projection saving)**.** *When agents have equal latent width and $r_\ell < C$, the learned projection component of a low rank latent handoff has lower asymptotic multiplication count than a full class-space decode of the same L positions.*

Proof. The link requires two low rank matrix products with order $O(Ldr_\ell)$. The output projection requires order $O(LCd)$. The former is smaller when $r_\ell < C$, up to constant factors. □

For ARC grids, C is small, so raw projection savings alone may be modest. The more important benefits are differentiability, preservation of uncertainty, and avoidance of an argmax bottleneck. The paper therefore treats latent communication as an information and optimization design, rather than claiming universal runtime superiority.

8.4 Sufficient Stability Condition

Let Φ be one complete collaboration-round map on the joint state $\mathcal{S}_t = (S_t, Z_t^{(1)}, \ldots, Z_t^{(M)}, Y_t)$.

Proposition 4 (Sufficient contraction condition)**.** *Suppose each active agent and link is Lipschitz continuous, the gated workspace update is nonexpansive, and the product of Lipschitz constants along every active sequential path is at most $\kappa < 1$. Then Φ is a contraction on a closed joint-state domain and repeated rounds converge to a unique fixed state.*

Proof. The Lipschitz constant of a composition is at most the product of component constants. Convex gated aggregation is nonexpansive under the stated assumption. Thus $\|\Phi(\mathcal{S}) - \Phi(\mathcal{S}')\| \le \kappa\|\mathcal{S} - \mathcal{S}'\|$. The Banach fixed-point theorem gives existence, uniqueness, and convergence. □

This proposition is a diagnostic sufficient condition, not a premise of MA-TRM training. The system remains well defined when the condition fails because recursion is explicitly bounded by $T_{\max}$. Spectral normalization or residual scaling may be used if oscillatory trajectories occur.

8.5 Verifier-Based Error Control

Assumption 1 (Cell calibration)**.** *For each cell q, the verifier confidence $c_{t,q}$ satisfies $\Pr(\hat{y}_{t,q} = y_q^\star \mid c_{t,q} = c) = c$ on the validation distribution.*

Proposition 5 (Expected Hamming error)**.** *Under cell calibration,*

$$\mathbb{E}[d_H(\hat{y}_t, y^\star) \mid \{c_{t,q}\}] = \sum_{q=1}^{L}(1 - c_{t,q}). \tag{57}$$

Therefore, a stopping rule that requires $\sum_q (1-c_{t,q}) \le \epsilon$ controls expected Hamming error by ϵ on the calibration distribution.

Proof. The Hamming error is the sum of cell error indicators. Linearity of expectation and calibration give (57). □

Disagreement is not itself a correctness certificate because agents can agree on the same wrong answer. It serves as a stability and epistemic diversity signal that complements calibrated verification.

9 Experimental Methodology

9.1 Research Questions

The evaluation should answer five questions:

RQ1: Does role-specialized recursion improve exact accuracy over parameter-matched and compute-matched TRM?

RQ2: Do latent links outperform decoded intermediate handoffs under matched topology and compute?

RQ3: Does disagreement-driven recursion improve accuracy per unit of computation?

RQ4: Do sparse routing and cell-level attention reduce cost without harming exact accuracy?

RQ5: Which collaboration topology provides the best accuracy, calibration, and efficiency trade-off?

9.2 Benchmarks

The primary benchmarks are Sudoku-Extreme, Maze-Hard, ARC-AGI-1, and ARC-AGI-2, following the structured reasoning setting established by HRM and TRM [2, 3]. Pencil Puzzle Bench may be added to test broader puzzle transfer, particularly because probabilistic TRM reports substantial gains there [6]. ARC tasks should use official splits and carefully documented augmentation.

9.3 Baselines

Required baselines include:

- released TRM configuration;
- TRM with more recursion rounds;
- TRM with a parameter increase equal to MA-TRM overhead;
- TRM with matched training and inference FLOPs;
- HRM where reproducible;
- probabilistic TRM at matched test-time trajectories;
- MA-TRM without learned links;
- MA-TRM with decoded communication;
- MA-TRM without private states;
- MA-TRM without disagreement-driven halting;
- dense and sparse MA-TRM variants.

9.4 Evaluation Regimes

To avoid confounding architecture with test-time search, every ARC result should be reported in four regimes:

(a) canonical single-pass inference;
(b) canonical adaptive recursion;
(c) augmented inference without voting;
(d) augmented multi-sample voting with the exact sample count reported.

Task identity embeddings should be evaluated as an explicit ablation. The main table should report results both with and without identity conditioning. Training data augmentation count,

voting count, and total inference FLOPs must appear beside accuracy.

9.5 Metrics

The primary quality metric is exact task accuracy. Secondary metrics are cell accuracy, verifier Brier score, expected calibration error, negative log likelihood, and correction rate after the first round. Efficiency metrics are parameter count, active parameter count, FLOPs, peak memory, wall-clock time, GPU-hours, mean rounds, mean active agents, mask density, and energy where measurable. The main combined metric is

$$\text{EfficiencyScore} = \frac{\text{ExactAccuracy}}{\log(1 + \text{InferenceFLOPs})}. \tag{58}$$

This score is supplementary. Raw accuracy and raw cost must remain visible.

9.6 Statistical Protocol

Use at least five seeds for Sudoku and Maze, and at least three seeds for full ARC experiments. Report mean, standard deviation, and bootstrap confidence intervals for exact accuracy. Compare paired outputs with McNemar's test when systems evaluate the same task set. Hyperparameters must be selected on validation data, and all final evaluation scripts should be frozen before test execution.

10 Implementation and Infrastructure

A faithful implementation can extend the released TRM code structure by adding agents, links, routing, workspace, and control modules. The initial development environment can use a single modern GPU. Full ARC reproduction should follow the original TRM scale, which reports multi-day training on four H100 GPUs in its public repository [19]. Activation checkpointing is recommended because unrolled collaboration stores multiple agent states even when parameter count is small.

The software stack should include Python, PyTorch, CUDA, Distributed Data Parallel, Hydra or an equivalent configuration system, and an experiment tracker. Deterministic and high-performance configurations should be separated. Every run must record the source commit, dataset hash, augmentation policy, random seed, model configuration, driver, CUDA, PyTorch, GPU model, and precision.

The implementation should expose a structured recursion trace containing active agents, router probabilities, per-cell masks, verifier confidence, disagreement, answer changes, and state norms. These traces are necessary to test whether the system performs genuine iterative correction rather than obtaining gains from additional parameters or voting.

11 Expected Failure Modes and Mitigations

11.1 Agent Collapse

All role adapters may learn nearly identical functions. The specialization loss, role-specific pretraining, router balancing, and representation analysis mitigate this risk. Collapse should be measured by centered kernel alignment and pairwise cosine similarity, rather than inferred from role names.

11.2 False Consensus

Agents may agree on an incorrect answer, making disagreement small. Calibrated verification, periodic global refresh, stochastic training corruptions, and optional probabilistic trajectories can provide independent safeguards. Disagreement must never be treated as a correctness certificate.

11.3 Routing Starvation

Top-k routing may prevent rarely selected agents from learning. Early training should use higher temperature, occasional random role activation, and a load-balancing objective. The verifier can be mandatory in every round.

11.4 Oscillatory Deliberation

The proposer and critic may alternate between incompatible answers. Answer-change penalties, residual damping, maximum-round limits, and spectral control of latent links can reduce oscillation.

11.5 Mask Lock-In

Cell attention may freeze an early mistake. Soft masks during training and periodic full-grid refresh prevent irreversible lock-in.

11.6 Benchmark Leakage and Identity Dependence

ARC augmentation and task identity embeddings can obscure generalization. The evaluation must disclose exact augmentation procedures, isolate identity features, and report canonical single-pass results [5].

12 Discussion

The proposed model treats collaboration as the recursion operator. Standard TRM computes repeated powers of one learned map, approximately f_θ^T. MA-TRM computes repeated compositions of role-conditioned maps and latent links:

$$\left(R_{Mi_1} A_M \circ \cdots \circ R_{12} A_1\right)^T. \tag{59}$$

The design hypothesis is that functional diversity can improve the quality of each recursive step. Controlled sharing prevents this diversity from requiring independent full networks.

Table 4. Minimum ablation matrix for a publication-quality evaluation. Every row should report exact accuracy, cell accuracy, calibration, parameters, FLOPs, mean rounds, active agents, memory, and GPU-hours.

Variant	Roles	Latent links	Private states	Disagreement halt	Sparse route	Cell attention	Purpose
TRM baseline	1	no	1	no	no	no	reproduce original architecture
Parameter-matched TRM	1	no	1	no	no	no	control for parameter count
Compute-matched TRM	1	no	1	no	no	no	control for recursive compute
MA-TRM shared backbone	4	yes	yes	no	no	no	isolate specialization
MA-TRM decoded messages	4	no	yes	no	no	no	compare communication medium
MA-TRM no private states	4	yes	no	no	no	no	test private memory
MA-TRM adaptive	4	yes	yes	yes	no	no	test disagreement recursion
MA-TRM sparse	4	yes	yes	yes	yes	no	test routing efficiency
MA-TRM-Lite	4	yes	yes	yes	yes	yes	full proposed model

Private states and the shared workspace play complementary roles. Private states allow agents to preserve evidence that is temporarily inconsistent with consensus. The shared workspace creates a common answer context and enables system-level credit. Too much consensus early can eliminate useful diversity, while too little consensus late can prevent stable output. The annealed objective in (50) makes this trade-off explicit.

Disagreement-driven recursion also changes the interpretation of depth. Depth becomes conditional on unresolved evidence rather than a fixed hyperparameter. Sparse routing makes agent count another adaptive axis, and cell attention makes spatial coverage adaptive. Together, the model allocates computation across three dimensions: rounds, roles, and cells.

The strongest version of the research claim should remain modest until experiments are complete. MA-TRM is expected to be most useful when tasks benefit from separable operations such as proposal, transformation, criticism, and verification. It may provide little advantage when one homogeneous operator already captures the task or when role supervision causes unnecessary inductive bias.

13 Limitations

This paper proposes an architecture and protocol, not a completed empirical study. The mathematical results are conditional accounting and stability statements, not proofs of improved reasoning accuracy. The contraction condition is sufficient but may not hold in trained networks. Verifier error control depends on calibration under the evaluation distribution. Low rank sharing can constrain role diversity too strongly. Sparse routing may reduce hardware efficiency when small irregular kernels dominate. Cell-level masks add control overhead that may exceed their savings on small grids.

The four agent roles are suitable for structured puzzles but may need revision for other domains. Distillation can inherit teacher errors. Deliberation can amplify correlated mistakes. ARC results are particularly sensitive to augmentation, task identity features, and test-time sampling, so claims of general reasoning should be made only after transparent ablations.

14 Conclusion

We introduced MA-TRM, a parameter-efficient framework in which tiny specialized reasoners refine a structured solution through latent collaboration. The architecture replaces homogeneous recursion with role-conditioned agents, residual latent links, private states, a shared workspace, disagreement-driven recursion, sparse routing, and cell-level attention. Four collaboration topologies and an inner-outer optimization procedure provide a unified model family. MA-TRM-Lite offers a concrete first implementation with a shared two-layer backbone and low rank specialization. The framework yields precise hypotheses about recursive specialization, communication, adaptive computation, and parameter sharing. Its scientific value will depend on parameter-matched, compute-matched, identity-aware, and augmentation-transparent evaluation.

A Recommended Training Schedule

A practical curriculum has four stages. Stage one reproduces TRM exactly. Stage two adds role adapters and trains role losses without recursion. Stage three trains latent links and the shared workspace while freezing most backbone parameters. Stage four enables sparse routing, cell attention, and halting, followed by low-rate joint fine-tuning. Suggested loss weights should be tuned by validation, starting with $\lambda_f = 1$, $\lambda_s = 0.5$, $\lambda_{\text{imp}} = 0.1$, $\lambda_v = 0.1$, $\lambda_{\text{spec}} = 0.01$, $\lambda_{\text{cons}} = 0.01$, $\lambda_{\text{bal}} = 0.01$, $\lambda_c = 10^{-3}$, $\lambda_h = 0.05$, and $\lambda_m = 10^{-3}$.

B Reproducibility Checklist

A complete release should include source code, environment files, exact dataset construction scripts, trained checkpoints, raw per-task predictions, all configuration files, evaluation scripts, and profiler traces. The paper should report total trainable parameters both with and without task identity embeddings, total forward evaluations, exact test-time sample count, and all stopping thresholds.

References

[1] F. Chollet, "On the measure of intelligence," arXiv:1911.01547, 2019.

[2] G. Wang, J. Li, Y. Sun, X. Chen, C. Liu, Y. Wu, M. Lu, S. Song, and Y. Abbasi-Yadkori, "Hierarchical reasoning model," arXiv:2506.21734, 2025.

[3] A. Jolicoeur-Martineau, "Less is more: Recursive reasoning with tiny networks," arXiv:2510.04871, 2025.

[4] X. Yang, J. Zou, R. Pan, R. Qiu, P. Lu, S. Diao, J. Jiang, H. Tong, T. Zhang, M. J. Buehler, J. He, and J. Zou, "Recursive multi-agent systems," arXiv:2604.25917, 2026.

[5] A. Roye-Azar, S. Vargas-Naranjo, D. Ghai, N. Balamurugan, and R. Amir, "Tiny recursive models on ARC-AGI-1: Inductive biases, identity conditioning, and test-time compute," arXiv:2512.11847, revised 2026.

[6] A. Sghaier, A. Parviz, and A. Jolicoeur-Martineau, "Probabilistic tiny recursive model," arXiv:2605.19943, 2026.

[7] A. Graves, "Adaptive computation time for recurrent neural networks," arXiv:1603.08983, 2016.

[8] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," Journal of Machine Learning Research, vol. 23, no. 120, pp. 1–39, 2022.

[9] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in Proceedings of the International Conference on Learning Representations, Toulon, France, 2017.

[10] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with Gumbel-Softmax," in Proceedings of the International Conference on Learning Representations, Toulon, France, 2017.

[11] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in Proceedings of the International Conference on Learning Representations, Virtual, 2022.

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems, Long Beach, CA, pp. 5998–6008, 2017.

[13] S. Bai, J. Z. Kolter, and V. Koltun, "Deep equilibrium models," in Advances in Neural Information Processing Systems, Vancouver, Canada, pp. 690–701, 2019.

[14] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang, and Z. Tu, "Deeply-supervised nets," in Proceedings of the International Conference on Artificial Intelligence and Statistics, San Diego, CA, pp. 562–570, 2015.

[15] L. Franceschi, P. Frasconi, S. Salzo, R. Grazzi, and M. Pontil, "Bilevel programming for hyperparameter optimization and meta-learning," in Proceedings of the International Conference on Machine Learning, Stockholm, Sweden, pp. 1568–1577, 2018.

[16] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proceedings of the International Conference on Learning Representations, New Orleans, LA, 2019.

[17] B. T. Polyak and A. B. Juditsky, "Acceleration of stochastic approximation by averaging," SIAM Journal on Control and Optimization, vol. 30, no. 4, pp. 838–855, 1992.

[18] P. Rauba, C. Fanconi, and M. van der Schaar, "Tiny autoregressive recursive models," arXiv:2603.08082, 2026.

[19] Samsung SAIL Montreal, "TinyRecursiveModels: Official code for less is more, recursive reasoning with tiny networks," GitHub repository, archived Apr. 2026. Available: `https://github.com/SamsungSAILMontreal/TinyRecursiveModels`

[20] RecursiveMAS, "RecursiveMAS: Official implementation for recursive multi-agent systems," GitHub repository, 2026. Available: `https://github.com/RecursiveMAS/RecursiveMAS`